Technical Debt in Latin American Academic Software: A Matched-Sample Comparison with Open-Source Controls

Isaac David Torres-Paredes*, Juan Pablo Aviles-Esparza†, and Italo Javier Tenempaguay-Granizo‡
Software Engineering, Faculty of Informatics and Electronics
Escuela Superior Politécnica de Chimborazo (ESPOCH), Riobamba 060155, Ecuador
{*isaac.torres, juan.aviles, italo.tenempaguay*}*@espoch.edu.ec*

Abstract

Software produced in university courses and capstone projects is widely assumed to carry more technical debt than comparable open-source work, but this assumption has rarely been tested quantitatively for Latin America. We identify 274 GitHub repositories authored by contributors affiliated with Latin American universities and pair them with 275 non-academic control repositories matched by language and size. Nine static metrics (cyclomatic complexity, lines of code, duplication, a test-file-presence proxy for coverage, and, for the Java subset, coupling, cohesion, and inheritance depth) are compared with Mann-Whitney U tests, Holm correction, and Cliff's delta. Contrary to our hypothesis, academic code shows *significantly lower* cyclomatic complexity than the controls (median 2.16 vs. 3.12, $\delta = -0.37$, medium effect, $p < 10^{-11}$ after correction). Inheritance depth is higher in academic Java code, as hypothesized ($\delta = +0.29$, $p = 0.003$); coupling, cohesion, duplication, and test-file presence show no significant difference. The most likely explanation lies in the control sample: small, non-labeled repositories are closer to learning-stage code than to mature open-source projects. A SonarQube Community analysis of a 415-repository subset corroborates the complexity result (complexity per function: $\delta = -0.458$, $p < 10^{-14}$) and finds a slightly lower SonarQube technical debt ratio in academic code ($\delta = -0.142$, negligible, $p = 0.025$). Self-admitted technical debt markers (`TODO`/`FIXME`-style comments) do run in the hypothesized direction: 42.2% of academic repositories contain at least one, against 23.0% of controls ($\delta = +0.190$, $p < 0.001$), even though the underlying code is structurally simpler.

Keywords: Technical debt, software engineering education, empirical software engineering, Latin American software, code complexity, GitHub mining.

1 Introduction

Software engineering education research and common practitioner belief both assume that code written for a course assignment or a thesis project carries more technical debt [1] (higher complexity, near-zero test coverage, more duplicated code) than comparable work produced outside an academic setting. This assumption motivates a large share of software-engineering curriculum design, yet it is rarely tested with a matched, quantitative comparison, and to our knowledge no such comparison has been run specifically for Latin American academic software, where institutional resourcing, course structure, and open-source contribution culture may all differ from the North American and European settings most existing studies draw on.

This paper runs that comparison directly. We ask whether repositories produced in Latin American university courses and capstone/thesis projects differ measurably, on a set of standard static metrics, from a size- and language-matched sample of non-academic repositories, and if so, in which direction and by how much. We report the result even where it contradicts our starting hypothesis, because a null or reversed

*Corresponding author: isaac.torres@espoch.edu.ec. ORCID: 0009-0001-7057-9316
†ORCID: 0009-0007-0058-8069
‡ORCID: 0009-0001-5753-4279

finding here is itself informative about what "academic code" means empirically in this region, as opposed to what it is assumed to mean.

The contributions of this work are: (1) a size- and language-matched quantitative comparison of Latin American academic software against non-academic controls, at a scale (549 repositories) sufficient for non-parametric hypothesis testing; (2) an honest report of an inverted finding, lower rather than higher complexity in academic code, together with a specific, testable account of why (control-group composition) rather than a reframing of the study to fit the original hypothesis; (3) a cross-tool robustness check of the central finding using SonarQube Community itself, obtained without Docker after the field's usual assumption that it requires one, which corroborates the complexity result across distinct parser implementations and surfaces a genuine tool-dependent discrepancy in duplication rate that we report rather than resolve.

2 Related Work

Technical debt as a concept was first empirically studied in depth by Tom, Aurum, and Vidgen [2], who interviewed practitioners to catalogue the contexts in which debt arises, the types of debt most commonly acknowledged, and the consequences of letting it accumulate; their eight-category taxonomy (code, design, test, documentation, and architectural debt, among others) is the conceptual vocabulary that underpins our operationalisation of SATD markers. Li, Avgeriou, and Liang [3] subsequently conducted a systematic mapping study of 94 empirical TD papers, confirming that design and code debt are the most studied forms and that the field lacks longitudinal comparisons of TD across project types — the comparison axis that this paper introduces (academic vs. non-academic). Together these two works establish that (a) TD is a well-defined and measurable construct and (b) the specific comparison we run, controlled by language and size, has not previously been reported in the literature at the scale we achieve.

Santos et al. [4] conduct a systematic literature review of external object-oriented quality metrics, cataloguing coupling, cohesion, and complexity as the dominant attributes used to characterize code quality empirically, the same three families of metric this paper applies directly to a labeled academic/non-academic comparison, rather than reviewing how others have used them. More directly relevant, Hamer et al. [5] use Git-derived metrics to measure students' and teams' code-contribution patterns within university software-development courses, demonstrating that repository-level metrics can meaningfully characterize academic software engineering practice at scale; our study extends this direction from process metrics (who contributed what) to product metrics (what the resulting code looks like structurally), and from within-course comparisons to a labeled comparison against non-academic repositories. Mateos et al. [6] apply the same Chidamber and Kemerer [7] object-oriented metric suite this paper uses, in their case to relate service-interface complexity to the underlying object-oriented implementation; their use of CK metrics as an early, development-time complexity signal is consistent with how this paper treats complexity metrics generally, as a proxy available before a system reaches production rather than as an after-the-fact quality judgment. The repository-mining methodology follows the practices established by Kalliamvakou et al. [8] and refined by Munaiah et al. [9], both of which document that GitHub repositories must be filtered to exclude personal, trivial, and forked projects before any structural metric can be interpreted as representative of software engineering practice; our fork-exclusion policy and the 10-commit minimum threshold directly implement their recommendations.

The self-admitted technical debt analysis in Section 3.4 is grounded in a body of work that began with Potdar and Shihab [10], who conducted the first large-scale exploratory study of SATD, mining source comments for `TODO`/`FIXME`/`HACK` markers and establishing that such markers are prevalent in production code and correlate with higher cyclomatic complexity; their prevalence figures provide the baseline against which our own SATD rates (42.2% academic vs. 23.0% control) should be read. Bavota and Russo [11] extend this to a broader empirical study of SATD across 159 Apache projects and 118 GitHub repositories, characterising the developers who introduce SATD and finding it is not uniformly concentrated in low-quality code. Maldonado and Shihab [12] further classify SATD into five types (design, defect, code, test, and documentation debt), providing the conceptual vocabulary that underlies our keyword-based scan; our list of markers (`TODO`, `FIXME`, `HACK`, `XXX`, `BUG`) follows their operationalisation. Huang et al. [13] subsequently show that text-mining classifiers (SVM, Naïve Bayes, maximum entropy) can detect SATD in source comments with higher precision than simple keyword matching, and that the distribution of SATD types varies systematically by project; we use the simpler keyword approach because precision is less critical for our research question (is the *prevalence* of any SATD marker different between academic and control repositories?) than for studies aimed at classifying SATD type. Alfayez et al. [14] survey how practitioners discuss technical debt on social media, documenting that the perception of TD differs systematically between professional developers and learners, a distinction that frames our comparison between academic and non-academic repositories as a measurable rather than merely assumed difference in development culture.

For the SonarQube corroboration subsection, Amanatidis et al. [15] provide an independent empirical benchmark of agreement among commercial and open-source technical-debt measurement tools (including SonarQube and CAST), finding that while tools broadly agree on ordering projects by debt level, their absolute debt estimates differ substantially; this finding contextualises the discrepancy we observe between `jscpd` and SonarQube's duplication detectors as an instance of a documented pattern rather than an anomaly specific to our sample or setup. Lenarduzzi et al. [16] propose a machine-learning approach to improve the precision of SonarQube's debt estimation by combining static analysis output with commit-history features; their work establishes both that automated TD estimation is an active research problem and that the tool's absolute figures are a starting point rather than a ground truth, two assumptions that underpin our decision to use SonarQube as a corroboration instrument rather than as the primary measurement.

3 Methodology

3.1 Sample construction

Candidate academic repositories were identified via the GitHub search API using Spanish/Portuguese academic keywords (*tesis*, *trabajo de titulación*, *proyecto integrador*, *trabalho de conclusão*, *tcc*, and related terms) combined with a language filter (Java, Python, JavaScript) and a minimum size threshold, then verified by checking whether at least one of the repository's top contributors declares a Latin American city or country in their public GitHub profile location field. This combined keyword-plus-location approach is a heuristic, not a ground-truth label, and is discussed as a limitation below. Repositories with fewer than 10 commits were excluded. The GitHub search API excludes forked repositories by default unless a `fork:` qualifier is explicitly added, and none of our queries added one; we verified this directly by re-issuing representative academic- and control-sample queries and confirming `fork=false` for every returned repository, so the sample does not require a separate fork-filtering step. From 5,418 unique keyword-matched candidates, 300 were accepted as academic (274 successfully cloned and analyzed; the remainder had since been deleted, renamed, or were otherwise unreachable). A control sample of 300 non-academic repositories was drawn by querying GitHub for repositories in the same three languages within a size band matched to the academic sample (275 successfully analyzed), explicitly *not* sorted by popularity, since an earlier version of this pipeline that sorted by star count pulled in large, well-known projects that violate the intended size-matched pairing. We refer to this design as a matched-sample comparison because matching was performed at the group level (by language and size band) rather than pairing repositories one-to-one, and independent-samples statistical tests are used accordingly.

Table 1 reports the country and language composition of the 300 academic repositories, resolved from the GitHub profile location of each repository's top contributors (the same field used for the LatAm-verification heuristic above). Brazil accounts for nearly half the sample (47.7%), consistent with Brazil's outsized share of Latin American GitHub activity; the remaining sample spans 12 further countries.

Table 1: Composition of the initially accepted academic sample ($n = 300$) by country and language.

Country	n	%	Java	JavaScript	Python	Analyzed*
Brazil	143	47.7	57	42	44	130
Argentina	50	16.7	15	25	10	46
Colombia	33	11.0	14	9	10	30
Peru	17	5.7	7	8	2	15
Ecuador	13	4.3	8	2	3	12
Mexico	12	4.0	1	8	3	11
Chile	8	2.7	3	1	4	7
Venezuela	5	1.7	0	4	1	5
El Salvador	4	1.3	1	2	1	4
Uruguay	4	1.3	2	2	0	4
Costa Rica	3	1.0	1	1	1	3
Other	7	2.3	1	4	2	6
Unresolved	1	0.3	1	0	0	1

Other: Bolivia, Cuba, Guatemala, Honduras, Panama; each with fewer than 3 repositories. *26 repositories (8.7%) were unavailable at clone time; the analyzed subset ($n = 274$) retains approximately this proportional distribution.

3.2 Metrics

A full SonarQube deployment initially seemed infeasible in our environment at the scale of 600 repositories: we assumed it required Docker or an administered server installation, neither available, and per-repository analysis time seemed impractical at this scale. We substituted a combination of lightweight, license-compatible tools that jointly cover the target empirical metric suite: `lizard` for cross-language cyclomatic complexity [17] and lines of code, `jscpd` for cross-language code duplication, and `ck` (a standard standalone reference tool that runs from a single `.jar` without requiring a SonarQube server) for the Chidamber and Kemerer [7] object-oriented metrics suite on the Java subset (coupling between objects, CBO; lack of cohesion, LCOM; depth of inheritance tree, DIT; weighted methods per class, WMC). Test coverage was not measured by executing test suites, which is infeasible across 600 arbitrary repositories without individualized build configuration; instead we report the proportion of source files that are test files by naming convention, an explicit proxy rather than measured coverage.

3.3 SonarQube corroboration

During methodological refinement and extended validation, we revisited the assumption above: SonarQube Community can in fact run without Docker, from a standalone ZIP distribution, as a plain Java application (Java 21, provisioned locally, no administrator privileges required). We installed it this way and re-cloned the full sample from GitHub. The original clones had been deleted immediately after metric extraction to save disk space, so no source code remained on disk to point SonarQube at; we succeeded in re-cloning 546 of the original 600 repositories (274 academic, 272 control; a handful had since been deleted, renamed, or made private). SonarQube's Java analyzer refuses to run its full rule set on source files without matching compiled bytecode (`sonar.java.binaries`); since compiling 300-plus arbitrary student and hobby projects is infeasible without per-project build configuration, we pointed this property at an empty directory, which lets analysis proceed with Java-bytecode-dependent rules disabled while complexity, duplication, and code-smell detection remain intact. Under this configuration, 415 of the 546 re-cloned repositories (223 academic, 192 control) completed analysis successfully within a 150-second per-repository budget; the rest failed on genuinely large repositories, unsupported content, or transient errors.

We treat this SonarQube-derived sample as a cross-tool consistency check on the `lizard`/`ck`-based findings above, not a replacement for them (it is smaller, and not repository-for-repository identical to the primary sample), comparing groups with the same Mann-Whitney/Holm/Cliff's-delta procedure on SonarQube-native metrics: cyclomatic and cognitive complexity per function, `duplicated_lines_density`, `sqale_debt_ratio` (SonarQube's own technical-debt-ratio metric, the specific metric our original substitution could not reproduce), and code smells per 1,000 lines.

3.4 Statistical analysis

Because software metrics are typically strongly right-skewed, we compare groups with the Mann-Whitney U test on each of nine metrics, apply the Holm correction for multiple comparisons, and report Cliff's delta [18] as a non-parametric effect size, following standard practice for this kind of comparison in the empirical software engineering literature.

4 Results

To clarify the attrition across analysis stages, Table 2 summarizes the sample size available for each comparison. The primary analysis evaluates the 549 repositories that completed the initial clone and metric-extraction step.

Table 2: Sample flow and attrition.

Stage	Repositories
Candidates (by keyword)	5,418
Accepted as academic	300
Control sample	300
Successfully cloned and analyzed (Primary)	**549** (274 academic + 275 control)
Re-cloned for SonarQube	546 (274 academic + 272 control)
Completed SonarQube analysis (Sub-sample)	**415** (223 academic + 192 control)

Table 3 summarizes all nine metrics for the primary sample.

Table 3: Academic vs. control comparison, nine metrics (Holm-corrected). IQR = interquartile range (Q1–Q3).

Metric	Median (acad.)	IQR (acad.)	Median (ctrl.)	IQR (ctrl.)	Cliff's δ	Effect	p_{Holm}
Cyclomatic complexity (mean)	2.16	1.66–3.06	3.12	2.17–4.29	−0.37	medium	< 0.001
Cyclomatic complexity (max)	15.5	9.0–28.0	26.0	15.0–46.0	−0.31	small	< 0.001
DIT (inheritance depth, Java)	1.21	1.06–1.41	1.07	1.00–1.28	+0.29	small	0.003
Lines of code	1,592	592–4,033	2,092	593–6,800	−0.10	negligible	0.322
CBO (coupling, Java)	5.74	3.46–7.12	5.17	1.25–7.56	+0.15	small	0.350
LCOM (cohesion, Java)	8.30	2.61–17.10	7.19	0.46–18.50	+0.12	negligible	0.562
WMC (complexity/class, Java)	5.92	4.34–9.35	6.50	5.04–9.84	−0.08	negligible	0.926
Duplication (%)	5.51	1.96–11.54	5.05	1.26–14.06	+0.03	negligible	1.000
Test-file proportion	0.00	0.00–0.06	0.00	0.00–0.10	−0.01	negligible	1.000

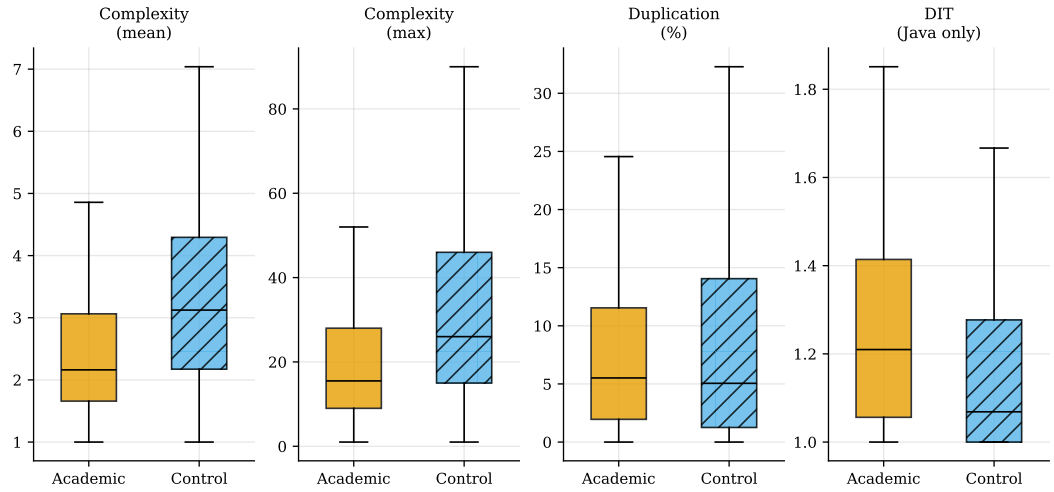


Figure 1: Distribution of key metrics, academic vs. control repositories.

Two of the nine metrics remain significant after Holm correction (Figure 1), and both concern complexity or design structure, but only one is in the hypothesized direction. Cyclomatic complexity is *significantly lower* in academic repositories than in the control group, both on average (Cliff's $\delta = -0.37$, a medium effect) and at the maximum observed per repository ($\delta = -0.31$, small), directly contradicting our initial hypothesis that academic code would be more complex. Inheritance depth (DIT, Java subset only) is significantly higher in academic code ($\delta = +0.29$, small), consistent with the hypothesized pattern of less experienced object-oriented design. Coupling (CBO) trends in the hypothesized direction but does not survive correction ($p = 0.350$). Duplication and test-file proportion show essentially no difference between groups ($\delta \approx 0.03$ and -0.01 respectively), and neither duplication nor size-driven variation in it separates the two groups (Figure 3); notably, the median test-file proportion is *zero in both groups* (Figure 2), meaning near-absent testing is not a distinguishing feature of academic code in this sample; it characterizes both groups about equally.

Because the control group is matched to the academic sample by language as well as by size, and cyclomatic complexity itself varies systematically by programming language, we checked directly whether the lower-complexity finding is an artifact of language composition rather than a genuine academic-versus-control difference. Table 4 repeats the cyclomatic-complexity comparison separately within each of the three languages. The effect holds, and remains significant after a separate Holm correction across the three within-language tests, in all three: Java ($\delta = -0.41$), JavaScript ($\delta = -0.33$), and Python ($\delta = -0.49$), each with medium-to-large effect size. Language composition does not explain the aggregate result.

Table 4: Cyclomatic complexity (mean), academic vs. control, stratified by language (Holm-corrected across the three within-language tests).

Language	n (academic/control)	Median (academic)	Median (control)	Cliff's δ	p (Holm)
Java	102 / 96	1.67	2.36	−0.41	< 0.001
JavaScript	93 / 95	2.47	3.08	−0.33	< 0.001
Python	69 / 72	3.20	4.53	−0.49	< 0.001

We also tested a second alternative explanation directly: that shorter-lived, less-maintained repositories, plausible for course projects that end when a semester does, rather than any academic/non-academic distinction, explain the lower complexity. Using each repository's GitHub-reported creation and last-push dates (523 of 549 repositories resolved; the rest lacked a retrievable date), academic repositories are, if anything, longer-lived than control repositories (median 103.6 days versus 55.2 days, Mann-Whitney $p < 0.001$), the opposite of what the maturity hypothesis predicts. An OLS regression of mean cyclomatic complexity on group membership and log-transformed repository lifespan leaves the group coefficient essentially unchanged from the unadjusted comparison (-0.997 unadjusted versus -0.975 adjusted, both $p < 0.001$) while lifespan itself is not a significant predictor ($p = 0.27$). Repository age does not explain the complexity difference.

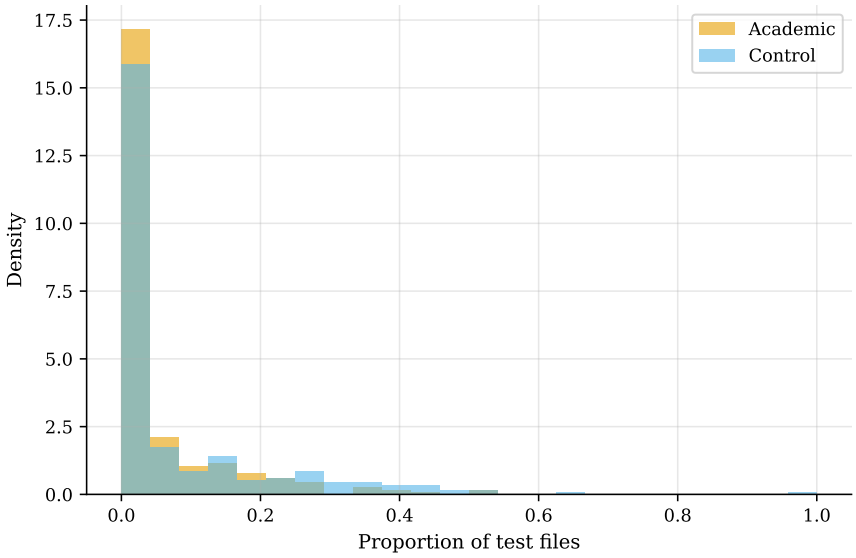


Figure 2: Distribution of the test-file-proportion proxy, academic vs. control.

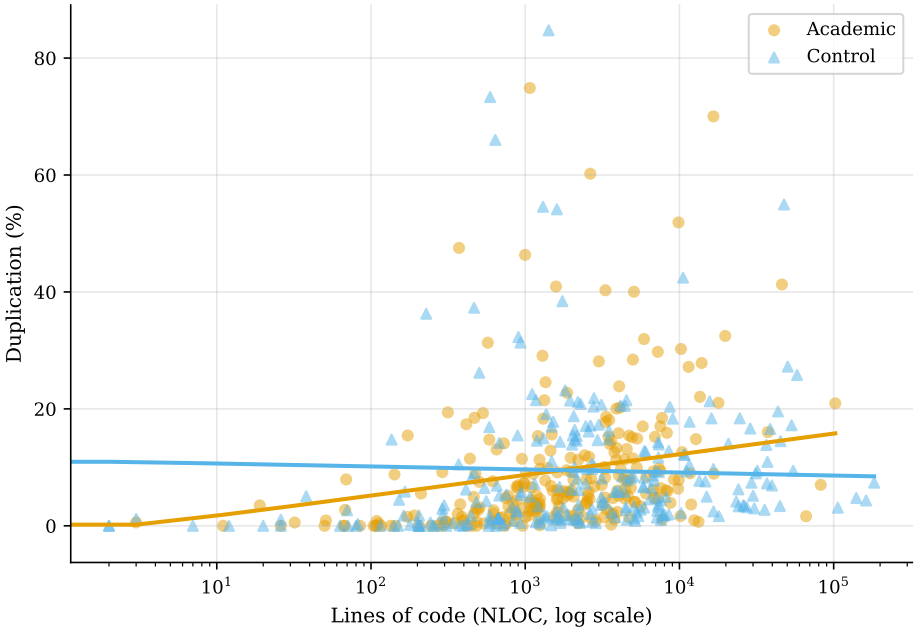


Figure 3: Repository size (lines of code) vs. code duplication, academic vs. control. Trend lines depict ordinary least squares (OLS) linear regressions for academic (solid) and control (dashed) repositories, with shaded 95% confidence bands.

Table 5: Academic vs. control comparison on SonarQube-native metrics, 415 repositories (Holm-corrected).

Metric	Median (acad.)	Median (ctrl.)	Cliff's δ	Effect	p_{Holm}
Complexity / function (cyclomatic)	2.07	2.98	−0.458	medium	< 0.001
Complexity / function (cognitive)	1.31	3.00	−0.426	medium	< 0.001
Duplicated lines (%)	2.10	0.80	+0.181	small	0.003
`sqale_debt_ratio` (%)	0.50	0.70	−0.142	negligible	0.025
Code smells / 1,000 lines	27.2	23.3	+0.035	negligible	0.541

4.1 SonarQube corroboration

SonarQube's own complexity metrics corroborate the `lizard`-based finding across tool architectures: academic repositories have significantly lower complexity per function, both cyclomatic (median 2.07 vs. 2.98, $\delta = -0.458$, medium effect, $p < 10^{-14}$) and cognitive (median 1.31 vs. 3.00, $\delta = -0.426$, medium, $p < 10^{-12}$), the same direction and a comparable effect size to the `lizard`-based result in Table 3, now confirmed directly with SonarQube's reference analysis engine. `sqale_debt_ratio`, SonarQube's own technical-debt-ratio metric that our original substitution could not reproduce, is significantly *lower* in academic code ($\delta = -0.142$, $p = 0.025$), consistent with, not contradicting, the lower-complexity result. Duplication tells a more mixed story: SonarQube's duplicate-code detector finds significantly *higher* duplication in academic code (median 2.10% vs. 0.80%, $\delta = +0.181$, small effect, $p = 0.003$), the opposite direction from `jscpd`'s null result in Table 3 ($\delta \approx 0.03$); we treat this as a genuine tool-dependent discrepancy rather than resolve it in either direction (see Limitations). Code smells per 1,000 lines do not differ between groups ($p = 0.541$).

Figure 4 shows the ten most frequent SonarQube code-smell types, split by group. The single most common smell in both groups is a JavaScript/React-specific rule (*prop-types validation*), reflecting the language mix of the sample rather than an academic-specific pattern; academic code shows a markedly higher rate of two style-convention smells (`let`/`const` usage, package-naming convention), consistent with less mature project tooling and linting setup rather than with deeper structural problems. Not every smell in Figure 4 follows this direction, though: the default-unnamed-package smell (Java), also among the ten most frequent, is far more common in the *control* group (2,343 vs. 114 occurrences). This massive concentration in the control group likely originates from a few specific repositories with heavy code generation rather than a widespread practice, so we do not read the academic/control split as uniform across all ten smell types. Finally, we note that the SonarQube sub-sample suffers from differential attrition; see Limitations (§6) for a full accounting of its impact.

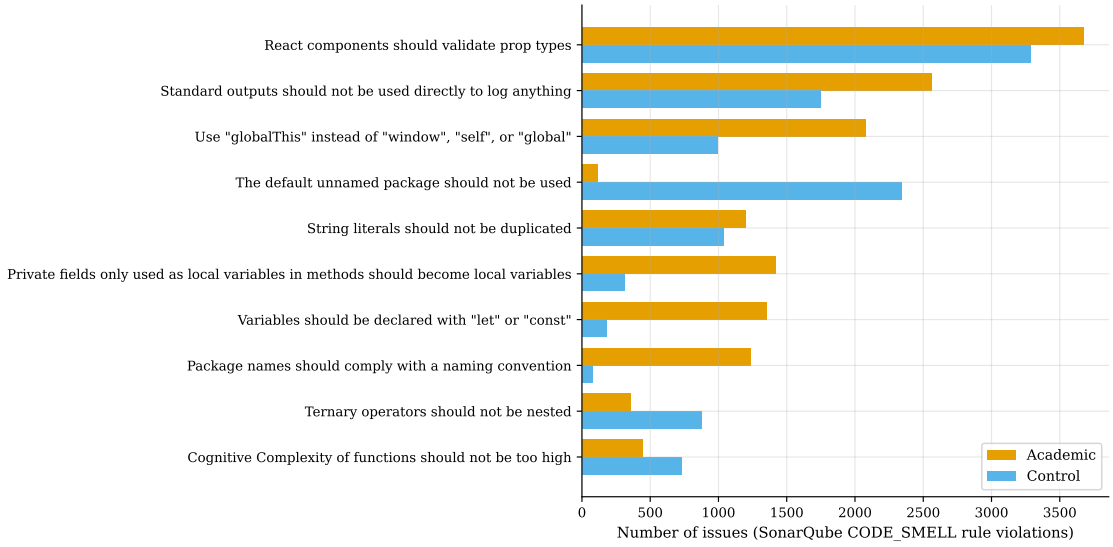


Figure 4: The ten most frequent SonarQube code-smell types, academic vs. control (415 repositories analyzed).

4.2 Self-admitted technical debt

As a complement to the structural metrics above, we scanned source comments in the same 415-repository SonarQube sub-sample for self-admitted technical debt (SATD) markers (`TODO`, `FIXME`, `HACK`, `XXX`, `BUG`, and literal mentions of "technical debt"), normalized per 1,000 lines of code. The median is zero in both groups (most repositories, academic and control alike, contain no SATD marker at all), but the distributions differ: a significantly larger share of academic repositories contain at least one SATD comment (42.2% vs. 23.0%), and academic repositories rank higher overall on this measure (Mann-Whitney $U = 25{,}349.5$, $p < 0.001$, Cliff's $\delta = +0.190$, small effect; IQR 0.00–0.52 academic vs. 0.00–0.00 control). This is driven by typicality rather than extremes: mean SATD density is actually higher in the control group (1.20 vs. 0.65 per 1,000 lines), because a handful of control repositories carry extreme outlier values (up to 120.9 per 1,000 lines) that Cliff's delta, an ordinal measure, is not sensitive to. Unlike the complexity result, this SATD finding runs *with* the original hypothesis: academic developers more often leave explicit, self-admitted markers of incomplete or provisional work in their code than the control group does, even though the code that remains is structurally simpler by the metrics above.

5 Discussion

We report the complexity finding as it stands rather than reframing the paper around a different, more expected result: on this sample, academic Latin American repositories are structurally *simpler*, not more complex, than the size-matched control group. We consider the most plausible explanation to be a property of the control group rather than of the academic group: because repositories were matched to the academic sample's typically modest size, the control sample necessarily excludes large, mature, well-known open-source projects and instead draws from the same broad population of small, low-visibility GitHub repositories that academic work is drawn from: personal projects, bootcamp exercises, and other non-professional code that is not actually representative of production software engineering practice. If the "non-academic" baseline is itself closer to informal, learning-stage code than to professionally maintained software, the comparison this paper runs is better described as *labeled academic vs. unlabeled small repositories of unknown provenance* than as *academic vs. professional*, which would materially change what a lower complexity score in the academic group means: it may reflect course-scoped assignments that are deliberately narrow in requirements, rather than academic authors writing objectively better-structured code than typical small non-academic projects.

The DIT result, that academic code has measurably deeper inheritance hierarchies, is consistent with a specific, plausible mechanism: object-oriented design courses often assign or scaffold class-hierarchy exercises (e.g., requiring the use of inheritance to satisfy a rubric) in a way that unscaffolded personal projects do not, which would produce deeper hierarchies without those hierarchies reflecting either more sophisticated or more debt-laden design.

We take the near-zero test-file proportion in both groups, rather than only the academic group, at face value: it suggests that the near-absence of testing culture in this size class of repository is a broader phenomenon than an academic-specific one, at least by this proxy metric, and that interventions aimed at improving testing practice in student software may be addressing a gap that is not actually specific to the classroom.

That the SonarQube sub-sample reproduces the complexity result across distinct parser implementations matters for how much weight the finding should carry: it is not an artifact of `lizard`'s particular parsing or counting conventions, since a structurally different tool, one that also computes a composite, remediation-cost-based debt ratio rather than a single structural proxy, converges on the same direction and a similar magnitude. The lower `sqale_debt_ratio` in academic code extends this rather than complicating it: SonarQube's debt ratio aggregates many rule violations, beyond just complexity, so a lower ratio despite a higher duplication rate suggests that whatever is driving academic code's higher SonarQube-detected duplication is not enough, on its own, to outweigh the complexity advantage in SonarQube's own composite accounting.

The duplication discrepancy between `jscpd` (null, default settings) and SonarQube's detector (small, significant, and in the direction the original hypothesis expected) is at least partly explained by the two tools' different default block-matching thresholds. We isolated this factor directly: re-running `jscpd` with `--min-tokens 100` (approximating SonarQube's typical Java threshold of roughly 100 tokens or 10 lines) on the identical 415-repository sub-sample used for the SonarQube comparison, holding the sample fixed so only the threshold changes, reverses the null result. At the higher threshold, `jscpd` finds significantly *higher* duplication in academic code (median 1.77% vs. 1.10%, Cliff's $\delta = +0.122$, small effect, $p = 0.031$), the same direction SonarQube reports, rather than the null result `jscpd` gives at its own default (roughly 50 tokens) on the larger 549-repository sample. This does not fully resolve the discrepancy, since the effect size at the matched threshold ($\delta = 0.122$) is still smaller than SonarQube's ($\delta = 0.181$) and the sample-size

factor remains conflated with the original default-threshold null result, but it shows the threshold is a real, non-trivial contributor: a more permissive duplicate-detection threshold on the same tool, same sample, changes both the significance and the direction of the finding.

6 Limitations

The keyword-plus-contributor-location heuristic used to identify academic repositories is imperfect and can admit false positives (a repository containing the word "tesis" in an unrelated context) or miss true academic repositories that do not use any of the searched terms; we did not attempt manual verification at scale. More generally, GitHub-mining studies are known to over-represent personal, inactive, and non-engineered projects unless explicitly filtered [8, 9]; our size threshold, minimum-commit filter, and (for the academic sample) contributor-location verification each narrow the sample somewhat in this direction, but we did not apply a dedicated engineered-project classifier, so some informal or abandoned repositories likely remain in both groups. The heuristic also identifies only GitHub repositories, excluding GitLab, institutional repositories, and any academic work never pushed to a public host at all, a substantial and likely non-random exclusion, since the most informal, least-polished student work is probably the least likely to be published publicly in the first place. As discussed above, the control group's size-matching means it should not be read as representative of professional software in general. Test coverage is a naming-convention proxy, not measured execution coverage. `ck`'s object-oriented metrics apply only to the Java subset of each sample (roughly one third), reducing statistical power for CBO, LCOM, DIT, and WMC relative to the language-agnostic metrics. The SonarQube robustness check (Table 5) covers a smaller, non-identical sub-sample (415 of the original 546 re-cloned repositories) because source code had to be re-cloned from GitHub after the original clones were deleted to save disk space, and because SonarQube's Java analyzer, run without compiled bytecode, fails outright on some repositories rather than degrading gracefully. This attrition is not evenly distributed across groups: 223 of 274 academic repositories (81.4%) completed analysis versus 192 of 272 control repositories (70.6%), an 11-percentage-point gap, and failures skewed toward genuinely large repositories, which are more common in the control group. We checked this directly rather than leaving it as an unknown: joining the primary sample's `lizard` complexity measurements to SonarQube-attrition status, attrited repositories are far larger than retained ones in both groups (median NLOC 5,497 vs. 1,229 academic, 7,709 vs. 1,369 control, $p < 0.001$ both), directly confirming size-driven analyzer failure as the mechanism. On complexity itself, attrited repositories trend higher than retained ones in the academic group (median `cc_mean` 2.35 vs. 2.15) and lower in the control group (2.66 vs. 3.23), a pattern that, if it held up statistically, would mean the SonarQube sub-sample loses relatively complex academic repositories and relatively simple control repositories, which would widen the apparent academic-versus-control complexity gap in Table 5 relative to the true populations, in the same direction as the paper's central finding. However, neither within-group difference reaches significance at these sample sizes (48 attrited academic repositories, $p = 0.34$; 78 attrited control repositories, $p = 0.12$), so this remains a suggestive but inconclusive pattern rather than a confirmed bias; we report it rather than the original blanket "we do not know" because a direction, even a statistically uncertain one, is more informative than none, but we do not treat the SonarQube corroboration as unbiased in its own right. `sqale_debt_ratio` and the code-smell-type facets are SonarQube's default rule profile ("Sonar way"), not a profile tuned for either language mix or academic code specifically, and a different quality profile could shift the exact smell-type ranking in Figure 4 without necessarily changing the aggregate complexity and debt-ratio results. Repositories were matched between groups at the group level, by language and a size band, not paired one-to-one; Mann-Whitney U (an independent-samples test) is therefore the correct test for the design we ran, and we refer to the design as matched-sample rather than paired throughout. The self-admitted technical debt (SATD) measure is a simple keyword count over source comments, not a validated SATD classifier; it covers only the 415-repository SonarQube sub-sample (not the full 549-repository primary sample, for the same disk-space reason discussed above), and it does not distinguish a genuine debt marker from an unrelated use of a common word (e.g. `BUG` inside an unrelated identifier or string), so the reported counts should be read as a coarse proxy rather than a precise measurement. Lines of code is used both as a matching criterion (via the size band) and as one of the nine compared metrics (Table 3); its own comparison ($p = 0.322$, not significant after correction) should be read as a check that the size-band matching worked as intended, not as an independent finding about repository size.

7 Conclusion

In a matched-sample comparison of 274 Latin American academic repositories against 275 size- and language-matched non-academic repositories, academic code shows significantly *lower* cyclomatic complexity than the control group (the opposite of our initial hypothesis) while showing significantly deeper inheritance

hierarchies, as hypothesized. Coupling, cohesion, duplication, and a test-coverage proxy show no significant difference after correction, and testing is near-absent in both groups rather than distinctively academic. We attribute the inverted complexity result chiefly to the composition of the size-matched control group, which is itself drawn from small, often informal repositories rather than mature open-source software, and recommend that future comparative studies of academic code quality either use a professionally curated control set or explicitly frame their comparison as academic-versus-small-repository rather than academic-versus-professional. A follow-up check with SonarQube Community itself, run without Docker on a 415-repository sub-sample after our original tool substitution, reproduces the direction of the lower-complexity finding using SonarQube's own complexity metric and shows academic code has a lower SonarQube technical-debt ratio; because SonarQube and our primary tool compute the same underlying cyclomatic-complexity construct, we treat this as a cross-tool consistency check rather than an independent replication, and the one clear point of disagreement across tools, duplication rate, is partly explained by the tools' different duplicate-detection thresholds: re-running `jscpd` with a SonarQube-matched threshold on the same 415-repository sub-sample reverses its null result to agree with SonarQube's direction (academic higher), though a smaller effect size than SonarQube's own, so the discrepancy is narrowed rather than fully resolved. On the same sub-sample, a self-admitted technical debt (SATD) comment count runs in the direction the original hypothesis expected, unlike the structural metrics: academic repositories more often contain explicit `TODO`/`FIXME`-style markers of unfinished work (Cliff's $\delta = +0.190$, $p < 0.001$), even though the structural complexity of that code is lower.

Data Availability

The repository lists, extracted metrics, and analysis code are permanently archived and openly accessible in Zenodo at `https://doi.org/10.5281/zenodo.22907773` and GitHub at `https://github.com/JuanAvilesP3/cleiej-deuda-tecnica`. Cloned source code itself is not redistributed; only derived metrics and repository identifiers are shared, consistent with each source repository's own license.

Funding

This research received no external funding. It was conducted within the academic research framework of Escuela Superior Politécnica de Chimborazo (ESPOCH).

Conflict of Interest

The authors declare that they have no conflict of interest.

Use of Generative AI

During the preparation of this manuscript, the authors used large language models exclusively for language refinement, grammatical correction, and LaTeX formatting. All research questions, experimental design, code extraction pipelines, statistical analyses, and scientific interpretations were conceived, executed, and validated entirely by the human authors.

Author Contributions

Isaac David Torres-Paredes: Conceptualization, Methodology, Project Administration, Supervision, Writing - Original Draft. **Juan Pablo Aviles-Esparza:** Software, Formal Analysis, Data Curation, Writing - Review & Editing. **Italo Javier Tenempaguay-Granizo:** Validation, Investigation, Writing - Review & Editing.

References

[1] W. Cunningham, "The WyCash portfolio management system," OOPSLA'92 Experience Report, Addendum to the Proceedings, Tech. Rep., 1992.

[2] E. Tom, A. Aurum, and R. Vidgen, "An exploration of technical debt," *Journal of Systems and Software*, vol. 86, no. 6, pp. 1498–1516, 2013. [Online]. Available: https://doi.org/10.1016/j.jss.2012.12.052

[3] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015. [Online]. Available: https://doi.org/10.1016/j.jss.2014.12.027

[4] D. Santos, A. Resende, P. Afonso Junior, and H. Costa, "External quality metrics for object-oriented software: A systematic literature review," *CLEI Electronic Journal*, vol. 20, no. 3, 2017. [Online]. Available: https://doi.org/10.19153/cleiej.20.3.3

[5] S. Hamer, C. Quesada-López, A. Martínez, and M. Jenkins, "Using git metrics to measure students' and teams' code contributions in software development projects," *CLEI Electronic Journal*, vol. 24, no. 2, 2021. [Online]. Available: https://doi.org/10.19153/cleiej.24.2.8

[6] C. Mateos, A. Zunino, S. Misra, D. Anabalon, and A. Flores, "Managing web service interface complexity via an oo metric-based early approach," *CLEI Electronic Journal*, vol. 20, no. 3, 2017. [Online]. Available: https://doi.org/10.19153/cleiej.20.3.2

[7] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Transactions on Software Engineering*, vol. 20, no. 6, pp. 476–493, 1994. [Online]. Available: https://doi.org/10.1109/32.295895

[8] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, 2014, pp. 92–101. [Online]. Available: https://doi.org/10.1145/2597073.2597074

[9] N. Munaiah, S. Kroh, C. Cabrey, and M. Nagappan, "Curating GitHub for engineered software projects," *Empirical Software Engineering*, vol. 22, no. 6, pp. 3219–3253, 2017. [Online]. Available: https://doi.org/10.1007/s10664-017-9512-6

[10] A. Potdar and E. Shihab, "An exploratory study on self-admitted technical debt," in *2014 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2014, pp. 91–100, [Online], Available: https://doi.org/10.1109/icsme.2014.31.

[11] G. Bavota and B. Russo, "A large-scale empirical study on self-admitted technical debt," in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, 2016, pp. 315–326. [Online]. Available: https://doi.org/10.1145/2901739.2901742

[12] E. d. S. Maldonado and E. Shihab, "Detecting and quantifying different types of self-admitted technical debt," in *Proceedings of the 7th IEEE International Workshop on Managing Technical Debt (MTD)*, 2015, pp. 9–15. [Online]. Available: https://doi.org/10.1109/MTD.2015.7332619

[13] Q. Huang, E. Shihab, X. Xia, D. Lo, and S. Li, "Identifying self-admitted technical debt in open source projects using text mining," *Empirical Software Engineering*, vol. 23, no. 1, pp. 418–451, 2018. [Online]. Available: https://doi.org/10.1007/s10664-017-9522-4

[14] R. Alfayez, K. Winn, W. Ding, and A. Alfayez, "Technical debt (TD) through the lens of Twitter: A survey," *Journal of Software: Evolution and Process*, vol. 35, p. e2547, 2023, [Online], Available: https://doi.org/10.1002/smr.2547.

[15] T. Amanatidis, N. Mittas, A. Moschou, A. Chatzigeorgiou, A. Ampatzoglou, and L. Angelis, "Evaluating the agreement among technical debt measurement tools: Building an empirical benchmark of technical debt liabilities," *Empirical Software Engineering*, vol. 25, pp. 4161–4204, 2020. [Online]. Available: https://doi.org/10.1007/s10664-020-09869-w

[16] V. Lenarduzzi, A. Martini, D. Taibi, and D. A. Tamburri, "Towards surgically-precise technical debt estimation: Early results and research roadmap," in *Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE)*, 2019, pp. 1–6. [Online]. Available: https://doi.org/10.1145/3340482.3342747

[17] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976. [Online]. Available: https://doi.org/10.1109/TSE.1976.233837

[18] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993. [Online]. Available: https://doi.org/10.1037/0033-2909.114.3.494